49

Sun 的网络文件系统（NFS）

分布式客户端/服务器（client/server）计算最早的用途之一，就是分布式文件系统（distributed file system）。在这样的环境里，有许多台客户端机器和一台（或少数几台）服务器；服务器把数据存放在自己的磁盘上，客户端则用格式规范的协议消息（protocol message）请求数据。图 49.1 展示了基本的结构。

从图中可以看到，磁盘在服务器上，客户端通过网络发送消息，访问这些磁盘上的目录（directory）和文件。为什么要费事采用这种结构呢？（也就是说，为什么不直接让客户端使用自己的本地磁盘？）主要是因为这种结构便于客户端之间**共享**（sharing）数据。如果你在一台机器（客户端 0）上访问了某个文件，之后又换到另一台（客户端 2），你看到的文件系统（file system）是一样的。你的数据自然而然地在这些不同的机器之间共享。第二个好处是**集中管理**（centralized administration）；例如，备份文件只需在少数几台服务器上做，不必在众多客户端上逐台去做。另一个好处可能是**安全**（security）；把所有服务器都放在上了锁的机房里，可以避免某些类型的问题。

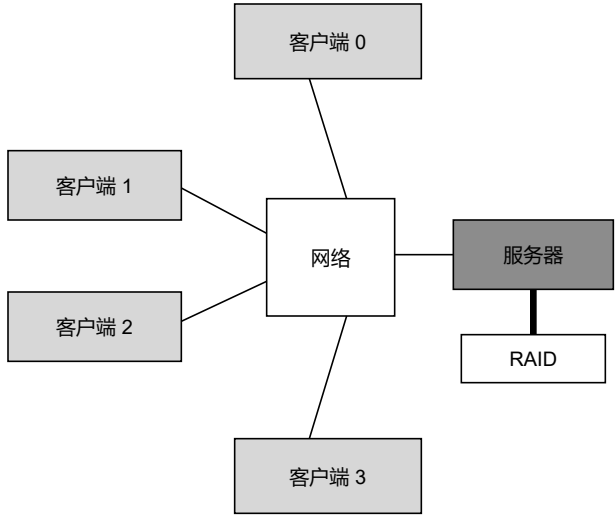


图 49.1：**一个通用的客户端/服务器系统**

关键问题：如何构建分布式文件系统
怎样构建分布式文件系统？需要考虑哪些关键方面？哪些地方容易出错？能从现有系统中学到什么？

49.1 一个基本的分布式文件系统

现在我们来研究一个简化的分布式文件系统的架构。简单的客户端/服务器分布式文件系统，组件比我们目前研究过的文件系统要多。在客户端一侧，客户端应用程序通过**客户端文件系统**（client-side file system）访问文件和目录。客户端应用程序向客户端文件系统发出**系统调用**（system call），例如 `open()`、`read()`、`write()`、`close()`、`mkdir()` 等，以访问存放在服务器上的文件。因此，在客户端应用程序看来，这个文件系统和本地（基于磁盘的）文件系统没有什么不同，只是性能上可能有差别；这样，分布式文件系统就提供了对文件的**透明**（transparent）访问。这是一个显而易见的目标；毕竟，谁会愿意用一个需要另一套 API、或者用起来很麻烦的文件系统呢？

客户端文件系统的作用，是完成这些系统调用所需的操作。例如，如果客户端发出 `read()` 请求，客户端文件系统可能会向**服务器端文件系统**（server-side file system，通常也叫**文件服务器**，file server）发送一条消息，要求读取某个特定的块（block）；文件服务器随后从磁盘或它自己的内存缓存（in-memory cache）中读出这个块，再发一条消息把请求的数据送回客户端。然后，客户端文件系统把数据复制到 `read()` 系统调用提供的用户缓冲区（user buffer）中，请求就完成了。注意，客户端之后再次 `read()` 同一个块时，这个块可能已经**缓存**（cached）在客户端的内存里，甚至在客户端的磁盘上；在最好的情况下，根本不需要产生网络流量。

从这个简单的概述中，你应该能体会到，客户端/服务器分布式文件系统里有两部分重要的软件：客户端文件系统和文件服务器。它们的行为共同决定了分布式文件系统的行为。现在是时候研究一个具体的系统了：Sun 的网络文件系统。

49.2 走近 NFS

最早的、也相当成功的分布式系统之一由 Sun Microsystems 开发，称为 Sun 网络文件系统（Sun Network File System，NFS）[S86]。在定义 NFS 时，Sun 采取了一种不同寻常的做法：它没有构建一个专有的封闭系统，而是开发了一个**开放协议**（open protocol），只规定客户端和服务器通信时使用的确切消息格式。不同的团队可以开发各自的 NFS 服务



图 49.2：**分布式文件系统的架构**

补充：服务器为什么会崩溃

在深入讨论 NFSv2 协议（protocol）的细节之前，你可能想知道：服务器为什么会崩溃？你也许猜到了，原因有很多。服务器可能只是遇到了（暂时的）**停电**（power outage）；只有恢复供电后，机器才能重新启动。服务器通常由几十万甚至上百万行代码组成；所以它们有 **bug**（即使是好软件，每一百行或一千行代码里也会有几个 bug），最终总会触发某个 bug，导致崩溃。它们还有内存泄漏（memory leak）；哪怕是很小的内存泄漏，也会让系统耗尽内存而崩溃。最后，在分布式系统（distributed system）中，客户端和服务器之间隔着网络；网络行为异常时，看起来就好像远程机器崩溃了，而实际上它只是暂时无法通过网络访问。例如，网络可能发生了**分区**（partitioned）：客户端和服务器都在正常工作，却无法相互通信。

器，在 NFS 市场上展开竞争，同时又保持互操作性（interoperability）。这一做法奏效了：如今有很多公司销售 NFS 服务器（包括 Oracle/Sun、NetApp [HLM94]、EMC、IBM 等），NFS 的广泛成功很可能要归功于这种“开放市场”的做法。

49.3 重点：简单、快速的服务器崩溃恢复

本章讨论的是经典的第 2 版 NFS 协议，即 NFSv2，它多年来一直是标准；升级到 NFSv3 时做了一些小改动，升级到 NFSv4 时协议则有较大规模的变化。不过，NFSv2 既精彩又让人头疼，因此是我们关注的重点。

在 NFSv2 中，协议设计的主要目标是简单、快速的服务器崩溃恢复。在多客户端、单服务器的环境里，这个目标很有道理；服务器宕机（或不可用）的每一分钟，都会让所有客户端机器（以及它们的用户）既不满意，也没法干活。所以，服务器怎样，整个系统就怎样。

49.4 快速崩溃恢复的关键：无状态

NFSv2 实现这个简单目标的办法，是设计一种我们称为**无状态**（stateless）的协议。按照设计，服务器不记录每个客户端那里正在发生的任何事情。例如，服务器不知道哪些客户端缓存了哪些块，不知道每个客户端当前打开了哪些文件，也不知道某个文件当前的文件指针位置，等等。简单地说，服务器不记录客户端在做什么；相反，协议的设计让每个协议请求都携带完成该请求所需的全部信息。如果你现在还不太明白，等下面更详细地讨论这个协议时，这种无状态的做法就会更好理解了。

来看一个**有状态**（stateful）而非无状态的协议的例子：`open()` 系统调用。给定一个路径名（pathname），`open()` 返回一个文件描述符（file descriptor），也就是一个整数。之后的 `read()` 或 `write()` 请求用这个描述符访问文件的各个块，如下面的应用程序代码所示（注意，为节省篇幅，省略了对系统调用的错误检查）：

现在设想，客户端文件系统打开文件的方式，是向服务器发送一条协议消息：“打开文件‘foo’，把描述符给我”。文件服务器随后在自己那一侧打开这个文件，把描述符发回客户端。之后读取时，客户端应用程序用这个描述符调用 `read()` 系统调用；客户端文件系

```
char buffer[MAX];
int fd = open("foo", O_RDONLY); // 获取描述符 "fd"
read(fd, buffer, MAX);     // 通过 "fd" 从 foo 读取 MAX 字节
read(fd, buffer, MAX);     // 再读 MAX 字节
...
read(fd, buffer, MAX);     // 再读 MAX 字节
close(fd);                 // 关闭文件
```

图 49.3：**客户端代码：读取文件**

统再把描述符放在消息里传给文件服务器，意思是："从我现在传给你的这个描述符所指的文件里，读一些字节"。

在这个例子中，文件描述符是客户端和服务器之间的一份**共享状态**（shared state，Ousterhout 称之为**分布式状态**，distributed state [O91]）。前面已经暗示过，共享状态（shared state）会让崩溃恢复（crash recovery）变得复杂。设想第一次读取完成之后、客户端发出第二次读取之前，服务器崩溃了。服务器重新启动并运行之后，客户端发出第二次读取。可惜，服务器根本不知道 `fd` 指的是哪个文件；这个信息是临时的（即只存在于内存中），服务器崩溃时就丢失了。为了处理这种情况，客户端和服务器必须执行某种**恢复协议**（recovery protocol）：客户端要确保自己的内存里保留足够的信息，以便告诉服务器它需要知道的东西（在这个例子里，就是文件描述符 `fd` 指向文件 `foo`）。

考虑到有状态的服务器还必须处理客户端崩溃，情况就更糟了。例如，设想一个客户端打开了一个文件，然后崩溃了。这次 `open()` 占用了服务器上的一个文件描述符；服务器怎么知道什么时候可以关闭某个文件呢？正常运行时，客户端最终会调用 `close()`，从而通知服务器应该关闭这个文件。然而，客户端崩溃时，服务器永远收不到 `close()`，因此必须察觉到客户端已经崩溃，才能关闭这个文件。

出于这些原因，NFS 的设计者决定采用无状态的方法：每个客户端操作都包含完成该请求所需的全部信息。不需要花哨的崩溃恢复；服务器只要重新开始运行就行了，客户端最坏也就是重试（retry）一次请求。

49.5 NFSv2 协议

下面来看 NFSv2 协议的定义。我们的问题很简单：

关键问题：如何定义无状态的文件协议

如何定义网络协议（network protocol），才能支持无状态的操作？显然，像 `open()` 这样有状态的调用不在考虑之列（因为它要求服务器记录打开的文件）；然而，客户端应用程序仍然会调用 `open()`、`read()`、`write()`、`close()` 等标准 API 来访问文件和目录。因此，更精确地说，问题是：如何定义一个协议，使它既是无状态的，又能支持 POSIX 文件系统 API？

 © 2008–25, ARPACI-DUSSEAU

理解 NFS 协议设计的一个关键，是理解**文件句柄**（file handle）。文件句柄（file handle）用来唯一地描述某个操作要作用的文件或目录；因此，很多协议请求（protocol request）里都带有文件句柄（file handle）。

文件句柄（file handle）可以看成由三个重要部分组成：卷标识符（volume identifier）、*inode* 号（inode number）和世代号（generation number）；这三者合在一起，构成了客户端想要访问的文件或目录的唯一标识。卷标识符（volume identifier）告诉服务器，请求针对的是哪个文件系统，因为一台 NFS 服务器可以导出不止一个文件系统；inode 号告诉服务器，请求要访问该分区中的哪个文件。最后，重用 inode 号时需要用到世代号（generation number）：每当重用一个 inode 号，服务器就把世代号加一，从而确保持有旧文件句柄（file handle）的客户端不会意外访问到新分配的文件。

图 49.4 概括了协议中一些重要的部分；完整的协议可以在别处找到（Callaghan 的书 [C00] 对 NFS 有出色而详细的概述）。

我们简要介绍一下协议中的重要组成部分。首先，LOOKUP 协议消息用于获取文件句柄（file handle），之后再用这个句柄访问文件数据。客户端传入一个目录的文件句柄（file handle）和要查找的文件名，服务器把该文件（或目录）的句柄及其属性传回客户端。

例如，假设客户端已经拿到了文件系统根目录（root directory，即 `/`）的目录文件句柄（directory file handle）。实际上，这个句柄是用 NFS **挂载协议**（mount protocol）得到的，客户端和服务器最初也正是靠它连接起来的；为简洁起见，这里不讨论挂载协议（mount protocol）。如果客户端上运行的应用程序打开文件 `/foo.txt`，客户端文件系统就会向服务器发送一个 lookup 请求，传入根目录的文件句柄（file handle）和名字 `foo.txt`；如果成功，就会返回 `foo.txt` 的文件句柄（file handle）和属性。

`NFSPROC_GETATTR`	文件句柄 返回：属性
`NFSPROC_SETATTR`	文件句柄，属性 返回：属性
`NFSPROC_LOOKUP`	目录的文件句柄，要查找的文件/目录名 返回：文件句柄，属性
`NFSPROC_READ`	文件句柄，偏移量，字节数 数据，属性
`NFSPROC_WRITE`	文件句柄，偏移量，字节数，数据 属性
`NFSPROC_CREATE`	目录的文件句柄，文件名，属性 文件句柄，属性
`NFSPROC_REMOVE`	目录的文件句柄，要删除的文件名 –
`NFSPROC_MKDIR`	目录的文件句柄，目录名，属性 文件句柄，属性
`NFSPROC_RMDIR`	目录的文件句柄，要删除的目录名 –
`NFSPROC_READDIR`	目录句柄，要读取的字节数，cookie 返回：目录项，cookie（用于获取更多目录项）

图 49.4：**NFS 协议：示例**

 © 2008–25, ARPACI-DUSSEAU

如果你好奇的话：属性就是文件系统为每个文件记录的元数据（metadata），包括文件创建时间、最后修改时间、大小、所有者和权限信息等字段，也就是对文件调用 `stat()` 时会得到的那类信息。

有了文件句柄（file handle），客户端就可以针对文件发出 READ 和 WRITE 协议消息，分别读写这个文件。READ 协议消息要带上文件句柄（file handle），以及文件内的偏移量（offset）和要读取的字节数。服务器据此就能完成读取：句柄告诉它要读哪个卷（volume）、哪个 inode，偏移量和字节数告诉它要读文件的哪些字节。随后，服务器把数据（以及最新的属性）返回给客户端；出错时则返回错误。WRITE 的处理与此类似，只是数据从客户端传到服务器，服务器返回的也只有一个成功码（以及最新的属性）。

最后一个值得一提的协议消息是 GETATTR 请求：它只是根据文件句柄（file handle）取出该文件的属性，其中包括文件的最后修改时间。下面讨论缓存（caching）时，我们会看到这个协议请求在 NFSv2 中为什么很重要（你能猜到为什么吗?）。

49.6 从协议到分布式文件系统

希望你现在已经大致明白，这个协议是如何借助客户端文件系统和文件服务器变成一个文件系统的。客户端文件系统记录打开的文件，并且一般会把应用程序的请求转换成相应的一组协议消息。服务器只负责响应协议消息，每条消息都包含完成请求所需的全部信息。

例如，我们来看一个读取文件的简单应用程序。图 49.5 给出了应用程序发出的系统调用，以及客户端文件系统和文件服务器为响应这些调用所做的事情。

关于这张图，有几点说明。第一，请注意客户端如何记录这次文件访问的所有相关**状态**（state），包括从整数文件描述符到 NFS 文件句柄（file handle）的映射，以及当前的文件指针。这样，客户端就能把每个读请求（你可能已经注意到，读请求并不显式指定从哪里开始读）转换成格式正确的读协议消息，准确地告诉服务器要读文件的哪些字节。读取成功后，客户端更新当前文件位置；后续的读取使用同一个文件句柄（file handle），但偏移量不同。

第二，你可能会注意到与服务器的交互发生在什么时候。第一次打开文件时，客户端文件系统发送一条 LOOKUP 请求消息。实际上，如果要遍历一个很长的路径名，例如 `/home/remzi/foo.txt`，客户端会发送三个 LOOKUP：一个在目录 / 中查找 `home`，一个在 `home` 中查找 `remzi`，最后一个在 `remzi` 中查找 `foo.txt`。

第三，你可能会注意到，每个服务器请求都包含完成该请求所需的全部信息。这个设计要点对于从服务器故障中平稳恢复至关重要，下面我们会更详细地讨论；它确保服务器不需要状态就能响应请求。

49.7 用幂等操作处理服务器故障

客户端向服务器发送消息后，有时收不到回复。没有回复的原因可能有很多。有时是消息被网络丢弃了；网络确实会丢失消息，所以请求或回复都可能丢失，客户端也就永远收不到响应。

客户端	服务器
fd = open(“/foo”, …); 发送 LOOKUP (rootdir FH, “foo”)	
	接收 LOOKUP 请求 在根目录中查找 “foo” 返回 foo 的 FH + 属性
接收 LOOKUP 回复 在打开文件表中分配文件描述符 把 foo 的 FH 存入表中 记录当前文件位置（0） 把文件描述符返回给应用程序	
read(fd, buffer, MAX); 用 fd 索引打开文件表 取得 NFS 文件句柄（FH） 用当前文件位置作为偏移量 发送 READ (FH, offset=0, count=MAX)	
	接收 READ 请求 用 FH 得到卷号/inode 号 从磁盘（或缓存）读取 inode 计算块的位置（利用偏移量） 从磁盘（或缓存）读取数据 把数据返回给客户端
接收 READ 回复 更新文件位置（+读取的字节数） 把当前文件位置设为 MAX 把数据/错误码返回给应用程序	
read(fd, buffer, MAX); 同上，只是 offset=MAX，并把当前文件位置设为 2*MAX	
read(fd, buffer, MAX); 同上，只是 offset=2*MAX，并把当前文件位置设为 3*MAX	
close(fd); 只需清理本地结构 释放打开文件表中的描述符 “fd” （无须与服务器通信）	

图 49.5：**读取文件：客户端和文件服务器的操作**

也有可能是服务器崩溃了，因此暂时不响应消息。过一会儿，服务器会重新启动并开始运行，但在此期间，所有请求都丢失了。在所有这些情况下，客户端都面临一个问题：服务器没有及时回复时，该怎么办？

在 NFSv2 中，客户端用一种统一而优雅的方式处理所有这些故障：它只是重试请求。具体来说，发送请求后，客户端设置一个定时器，在指定的时间之后触发。如果在定时器触发之前收到了回复，就取消定时器，一切正常。但如果定时器在收到任何回复之前就触发

提示：幂等性很强大

幂等性（idempotency）是构建可靠系统时一个很有用的性质。如果一个操作可以发出不止一次，处理这个操作的失败就容易得多：直接重试就行了。如果一个操作不是幂等（idempotent）的，日子就难过多了。

了，客户端就认为请求没有被处理，于是重新发送。如果服务器回复了，一切正常，客户端也就漂亮地解决了问题。

客户端之所以能简单地重试请求（不管故障是由什么引起的），是因为大多数 NFS 请求有一个重要性质：它们是**幂等的**（idempotent）。如果一个操作执行多次的效果和执行一次的效果相同，这个操作就称为幂等的。例如，往某个内存位置存一个值，存三次和存一次是一样的；所以“把值存入内存”是幂等操作。但如果把一个计数器加三次，结果就和只加一次不同；所以“计数器加一”不是幂等的。更一般地说，任何只读取数据的操作显然都是幂等的；而更新数据的操作，则必须仔细考虑才能确定它是否具有这个性质。

NFS 崩溃恢复设计的核心，就在于大多数常用操作的幂等性。LOOKUP 和 READ 请求显然是幂等的，因为它们只从文件服务器读取信息，不做更新。更有意思的是，WRITE 请求也是幂等的。例如，如果一次 WRITE 失败了，客户端可以直接重试。WRITE 消息包含数据、字节数，以及（重要的是）写入数据的确切偏移量。因此，它可以重复执行，而且我们知道，多次写入的结果和一次写入的结果相同。

这样，客户端就能用统一的方式处理所有超时（timeout）。如果一个 WRITE 请求只是丢失了（上面的情况 1），客户端会重试，服务器会执行写入，一切都会正常。如果发送请求时服务器恰好宕机，而发送第二个请求时服务器已经恢复运行，结果也一样，一切照常（情况 2）。最后，服务器可能实际上已经收到了 WRITE 请求，把数据写到了磁盘，并发送了回复。这个回复可能丢失（情况 3），同样会导致客户端重新发送请求。服务器再次收到请求时，只会做完全相同的事情：把数据写入磁盘，然后回复说已经写好了。如果客户端这次

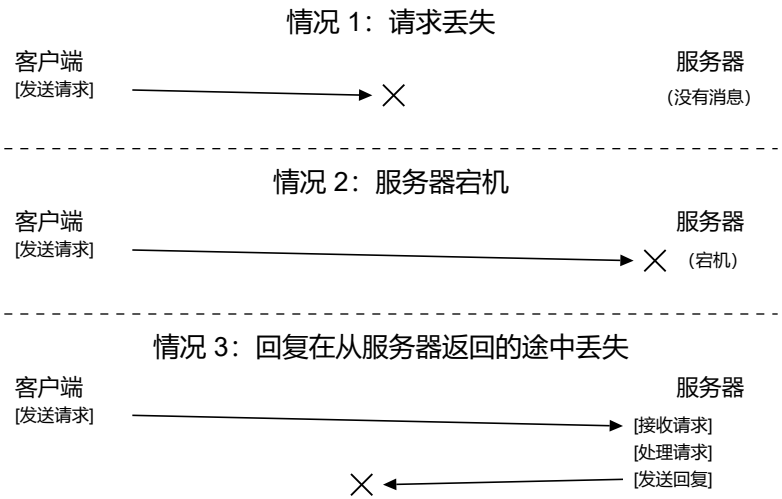


图 49.6：**三种丢失情况**

 © 2008–25, ARPACI-DUSSEAU

提示：完美是优秀的敌人（伏尔泰定律）
即使你设计了一个漂亮的系统，有时也不是所有的边界情况都能如你所愿。以上面的 mkdir 为例：可以重新设计 mkdir，赋予它不同的语义，从而让它变成幂等的（想想你会怎么做）；然而，何必费这个劲呢？NFS 的设计理念覆盖了大多数重要的情况，总体上让系统在故障处理方面的设计干净、简单。因此，接受生活并不完美、照样把系统构建出来，正是良好工程的标志。据说这条智慧出自伏尔泰，他说过："……一位聪明的意大利人说，'最好'是'好'的敌人"[V72]，因此我们称之为**伏尔泰定律**（Voltaire's Law）。

收到了回复，一切又恢复正常，这样客户端就用统一的方式同时处理了消息丢失和服务器故障。漂亮！

顺便说一句：有些操作很难做成幂等的。例如，试图创建一个已经存在的目录时，mkdir 请求会报告失败。因此，在 NFS 中，如果文件服务器收到 MKDIR 协议消息并成功执行，但回复丢失了，客户端可能会重试，结果收到失败，而这个操作其实第一次就成功了，只是重试时才失败。所以说，生活并不完美。

49.8 提高性能：客户端缓存

分布式文件系统有很多好处，但通过网络发送所有的读写请求，可能导致很大的性能问题：网络一般没那么快，与本地内存或磁盘相比尤其如此。于是又有一个问题：如何提高分布式文件系统的性能？

你读了上面小标题里那几个大字，大概能猜到，答案是客户端**缓存**（caching）。NFS 客户端文件系统会把从服务器读到的文件数据以及元数据缓存在客户端内存中。这样，第一次访问的开销很大（需要网络通信），而后续访问直接从客户端内存取数据，速度很快。

缓存也可以充当写操作的临时缓冲区（buffer）。客户端应用程序第一次写文件时，客户端先把数据缓冲在客户端内存中，和从文件服务器读到的数据放在同一个缓存里，之后再把数据写到服务器。这种**写缓冲**（write buffering）很有用，因为它把应用程序 `write()` 的延迟和实际的写性能分开了：应用程序对 `write()` 的调用立即成功，只是把数据放进了客户端文件系统的缓存；数据要到之后才会写到文件服务器。

所以，NFS 客户端缓存了数据，性能通常很好，我们就大功告成了，对吧？可惜，还不完全是。只要系统里有多个客户端缓存，加入缓存就会带来一个重大而有趣的挑战，我们称之为**缓存一致性问题**（cache consistency problem）。

49.9 缓存一致性问题

用三个客户端和一台服务器最能说明缓存一致性问题（cache consistency problem）。设想客户端 C1 读取了文件 F，并在本地缓存中保留了一份副本。现在设想另一个客户端 C2 覆盖写了文件 F，改变了它的内容；我们把文件的新版本称为 F（版本 2），即 F[v2]，旧版本称为 F[v1]，以便区分这两者（当然，文件名还是同一个，只是内容不同）。最后，还有第三个客户端 C3，它还没有访问过文件 F。

 © 2008–25, ARPACI-DUSSEAU

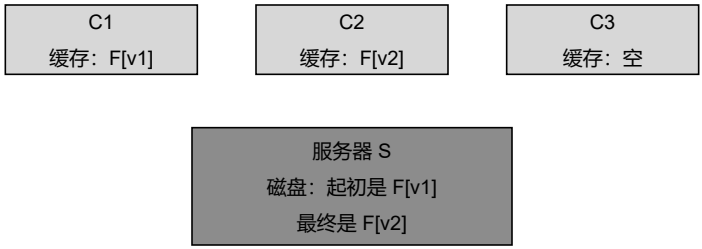


图 49.7：**缓存一致性问题**

你大概已经看出接下来的问题了（图 49.7）。实际上，这里有两个子问题。第一个子问题是，客户端 C2 可能会先把写操作缓冲在自己的缓存里一段时间，然后才传播给服务器；这时，F[v2] 还待在 C2 的内存里，其他客户端（比如 C3）对 F 的任何访问，取到的都是文件的旧版本（F[v1]）。因此，在客户端缓冲写操作，可能导致其他客户端拿到文件的过时版本，这未必是我们想要的；设想你登录机器 C2，更新了 F，然后登录 C3 去读这个文件，结果读到的却是旧副本！这肯定让人恼火。因此，我们把缓存一致性问题（cache consistency problem）的这一方面称为 **update visibility**：一个客户端的更新，什么时候才能被其他客户端看到?

缓存一致性（cache consistency）的第二个子问题是 **stale cache**：在这种情况下，C2 终于把写操作刷新到了文件服务器，于是服务器有了最新版本（F[v2]）。然而，C1 的缓存中仍然是 F[v1]；如果 C1 上运行的程序读取文件 F，它会得到过时的版本（F[v1]），而不是最新的副本（F[v2]），这（通常）不是我们想要的。

NFSv2 的实现用两种方式解决这些缓存一致性问题（cache consistency problem）。第一，为了解决 update visibility，客户端实现了有时称为**关闭时刷新**（flush-on-close，也叫 **close-to-open**）的一致性语义；具体来说，客户端应用程序写了一个文件并随后关闭它时，客户端会把所有更新，也就是缓存中的脏页（dirty page），刷新到服务器。有了关闭时刷新（flush-on-close）的一致性，NFS 就能确保之后另一个节点打开这个文件时，看到的是最新的版本。

第二，为了解决 stale cache 问题，NFSv2 客户端在使用缓存的内容之前，先检查文件是否发生了变化。具体来说，在使用某个缓存块（cached block）之前，客户端文件系统会向服务器发出 GETATTR 请求，获取文件的属性。重要的是，属性中包含文件在服务器上最后一次修改的时间；如果修改时间晚于文件被取到客户端缓存的时间，客户端就会让这个文件**失效**（invalidates），把它从客户端缓存中移除，从而确保后续的读取会去服务器取回文件的最新版本。反过来，如果客户端发现自己拥有的是文件的最新版本，它就会直接使用缓存的内容，从而提高性能。

Sun 最初的团队实现了这个 stale cache 问题的解决方案后，发现了一个新问题：NFS 服务器突然被大量 GETATTR 请求淹没了。有一条工程原则值得遵循：为**常见情况**（common case）而设计，让常见情况运行良好；在这里，虽然常见情况是文件只被单个客户端访问（也许是反复访问），但客户端总要向服务器发送 GETATTR 请求，以确认没有别人修改过这个

文件。于是，客户端不停地轰炸服务器，反复问“有人改过这个文件吗?”，而大多数时候根本没人改过。

为了（在一定程度上）补救这种情况，每个客户端都加上了**属性缓存**（attribute cache）。客户端在访问文件之前仍然会验证文件，但大多数时候只需查看属性缓存（attribute cache）来取得属性。某个文件的属性在第一次访问该文件时放入缓存，经过一段时间（比如 3 秒）后超时。因此在这 3 秒内，每次访问文件都会认定缓存的文件可用，于是直接使用，不必与服务器做任何网络通信。

49.10 评估 NFS 的缓存一致性

最后再谈几句 NFS 的缓存一致性（cache consistency）。加入关闭时刷新（flush-on-close）的行为是为了“说得通”，但它带来了一定的性能问题。具体来说，如果在客户端上创建了一个临时的或短命的文件，并且很快就删除了，它仍然会被强制写到服务器。更理想的实现，可能会把这类短命文件一直留在内存里，直到它们被删除，从而完全省掉与服务器的交互，也许还能提高性能。

更重要的是，在 NFS 中加入属性缓存（attribute cache）之后，你很难理解或推断自己拿到的究竟是文件的哪个版本。有时你会拿到最新版本；有时你会拿到旧版本，仅仅因为你的属性缓存（attribute cache）还没有超时，于是客户端乐于把内存里的东西给你。虽然这在大多数时候没问题，但它偶尔会（而且至今仍会!）导致奇怪的行为。

以上就是 NFS 客户端缓存（client caching）的怪异之处。它是一个有趣的例子：由实现细节来定义用户可观察到的语义，而不是反过来。

49.11 对服务器端写缓冲的影响

到目前为止，我们关注的都是客户端缓存（client caching），大多数有趣的问题也都出在这里。然而，NFS 服务器往往也是配置精良、内存充足的机器，所以它们同样要考虑缓存（caching）的问题。从磁盘读取数据和元数据时，NFS 服务器会把它们保留在内存中，之后再读这些数据（和元数据）就不用去磁盘了，这可能带来（小幅的）性能提升。

更耐人寻味的是写缓冲（write buffering）的情况。在写操作被强制写入稳定存储（stable storage，例如磁盘或其他某种持久设备）之前，NFS 服务器绝对不能对 WRITE 协议请求返回成功。虽然服务器可以在内存中放一份数据副本，但对 WRITE 协议请求向客户端返回成功，可能导致错误的行为；你能想明白为什么吗?

答案在于我们对客户端如何处理服务器故障所做的假设。设想客户端发出了下面这一系列写操作：

```
write(fd, a_buffer, size); // 用 a 填满第 1 块
write(fd, b_buffer, size); // 用 b 填满第 2 块
write(fd, c_buffer, size); // 用 c 填满第 3 块
```

这些写操作依次用一块 a、一块 b、一块 c 覆盖文件的三个块。因此，如果文件最初是这样的：

```
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
yyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy
zzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzzz
```

我们可能会预期，这些写操作完成后，最终结果是这样的：x、y、z 分别被 a、b、c 覆盖。

```
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb
ccccccccccccccccccccccccccccccccccccccccccccccccc
```

为了举例，假设客户端的这三次写操作以三条不同的 WRITE 协议消息发给了服务器。假设第一条 WRITE 消息被服务器收到并写入了磁盘，客户端也得知它成功了。现在假设第二次写入只是缓冲在内存里，服务器没等把它强制写入磁盘，就向客户端报告了成功；不幸的是，服务器随后在写盘之前崩溃了。服务器很快重新启动，并收到了第三个写请求，这次请求也成功了。

于是，在客户端看来，所有请求都成功了，可我们惊讶地发现文件内容是这样的：

```
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
yyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy <--- 糟了
ccccccccccccccccccccccccccccccccccccccccccccccccc
```

天哪！因为服务器在把第二次写入提交到磁盘之前就告诉客户端它成功了，文件里留下了一块旧数据；视具体应用而定，这可能是灾难性的。

为了避免这个问题，NFS 服务器必须先把每个写操作提交到稳定存储（stable storage，即持久存储），然后才能通知客户端成功；这样客户端就能察觉写入期间发生的服务器故障，并不断重试，直到最终成功。这样做可以确保我们永远不会得到像上面例子中那样相互混杂的文件内容。

这个要求给 NFS 服务器的实现带来的问题是：如果不精心处理，写性能可能会成为最主要的瓶颈。事实上，有些公司（例如 Network Appliance）成立时的目标很简单，就是造

补充：创新孕育创新

和许多开创性技术一样，NFS 的问世也需要其他一些基础性的创新来支撑它的成功。其中影响最持久的，大概是 Sun 引入的**虚拟文件系统**（Virtual File System，**VFS**）/ **Virtual Node**（**vnode**）接口，它让不同的文件系统可以方便地接入操作系统 [K86]。

VFS 层包含对整个文件系统执行的操作，比如挂载（mount）和卸载（unmount）、获取整个文件系统范围的统计信息，以及把所有脏的（尚未写出的）写操作强制写入磁盘。

vnode 层则由可以对文件执行的所有操作组成，比如 open、close、读、写等等。

要构建一个新的文件系统，只需定义这些“方法”；框架会处理其余的一切：把系统调用连接到具体的文件系统实现，集中执行所有文件系统共有的通用功能（例如缓存），从而让多个文件系统实现可以在同一个系统中同时运行。

虽然有些细节已经改变，但许多现代系统都有某种形式的 VFS/vnode 层，包括 Linux、各种 BSD、macOS，甚至 Windows（形式是可安装文件系统，即 Installable File System）。即使 NFS 对世界来说没那么重要了，支撑它的一些必要基础也会延续下去。

 © 2008–25, ARPACI-DUSSEAU

出写得快的 NFS 服务器；它们用的一个技巧是，先把写操作放进有电池备份的内存里，这样就能快速回复 WRITE 请求，既不必担心丢失数据，也不必付出立即写盘的代价；第二个技巧是采用专门设计的文件系统，让最终需要写盘时能快速写入 [HLM94, RO91]。

49.12 小结

我们介绍了 NFS 分布式文件系统。NFS 的核心思想是在服务器发生故障时实现简单、快速的恢复，并通过精心的协议设计达到了这个目的。操作的幂等性至关重要：客户端可以放心地重放（replay）失败的操作，不管服务器有没有执行过这个请求，重放都没有问题。

我们还看到了，在多客户端、单服务器的系统中引入缓存（caching）会让事情变得复杂。具体来说，系统必须解决缓存一致性问题（cache consistency problem），才能表现得合理；然而，NFS 解决这个问题的办法有点临时拼凑的味道，偶尔会导致可以观察到的怪异行为。最后，我们看到了服务器端缓存（server caching）可能很棘手：写到服务器的数据必须先强制写入稳定存储（stable storage），才能返回成功（否则数据可能丢失）。

还有一些显然很重要的问题我们没有讨论，尤其是安全。早期 NFS 实现的安全性出奇地宽松；客户端上的任何用户都可以相当轻易地冒充其他用户，从而访问几乎任何文件。后来集成了更严格的身份认证（authentication）服务（例如 Kerberos [NT94]），才弥补了这些明显的缺陷。

补充：NFS 的关键术语

- 在 NFS 中，实现快速、简单的崩溃恢复这一主要目标，关键在于设计一个**无状态**（stateless）的协议。崩溃之后，服务器可以很快重新启动并开始处理请求；客户端只需**重试**（retry）请求，直到成功为止。
- 让请求具有**幂等性**（idempotency）是 NFS 协议的核心之一。如果一个操作执行多次的效果和执行一次相同，这个操作就是幂等的。在 NFS 中，幂等性让客户端可以放心地重试，并把客户端重传丢失的消息和客户端处理服务器崩溃这两件事统一了起来。
- 出于性能考虑，需要客户端**缓存**（caching）和**写缓冲**（write buffering），但这也带来了**缓存一致性问题**（cache consistency problem）。
- NFS 的实现通过多种手段为缓存一致性（cache consistency）提供了一种工程上的解决方案：**关闭时刷新**（flush-on-close，也叫 **close-to-open**）的做法确保文件关闭时内容会被强制写到服务器，其他客户端因此能看到对它的更新。属性缓存（attribute cache）则降低了向服务器检查文件是否改变（通过 GETATTR 请求）的频率。
- NFS 服务器必须先把写操作提交到持久介质，然后才能返回成功；否则可能丢失数据。
- 为了支持把 NFS 集成到操作系统中，Sun 引入了 **VFS/vnode** 接口，使多种文件系统实现能够在同一个操作系统中共存。

 © 2008–25, ARPACI-DUSSEAU

参考文献

[AKW88] "The AWK Programming Language" by Alfred V. Aho, Brian W. Kernighan, Peter J. Weinberger. Pearson, 1988 (1st edition). *A concise, wonderful book about awk. We once had the pleasure of meeting Peter Weinberger; when he introduced himself, he said "I'm Peter Weinberger, you know, the 'W' in awk?" As huge awk fans, this was a moment to savor. One of us (Remzi) then said, "I love awk! I particularly love the book, which makes everything so wonderfully clear." Weinberger replied (crestfallen), "Oh, Kernighan wrote the book."*

[C00] "NFS Illustrated" by Brent Callaghan. Addison-Wesley Professional Computing Series, 2000. *A great NFS reference; incredibly thorough and detailed per the protocol itself.*

[ES03] "New NFS Tracing Tools and Techniques for System Analysis" by Daniel Ellard and Margo Seltzer. LISA '03, San Diego, California. *An intricate, careful analysis of NFS done via passive tracing. By simply monitoring network traffic, the authors show how to derive a vast amount of file system understanding.*

[HLM94] "File System Design for an NFS File Server Appliance" by Dave Hitz, James Lau, Michael Malcolm. USENIX Winter 1994. San Francisco, California, 1994. *Hitz et al. were greatly influenced by previous work on log-structured file systems.*

[K86] "Vnodes: An Architecture for Multiple File System Types in Sun UNIX" by Steve R. Kleiman. USENIX Summer '86, Atlanta, Georgia. *This paper shows how to build a flexible file system architecture into an operating system, enabling multiple different file system implementations to coexist. Now used in virtually every modern operating system in some form.*

[NT94] "Kerberos: An Authentication Service for Computer Networks" by B. Clifford Neuman, Theodore Ts'o. IEEE Communications, 32(9):33-38, September 1994. *Kerberos is an early and hugely influential authentication service. We probably should write a book chapter about it sometime…*

[O91] "The Role of Distributed State" by John K. Ousterhout. 1991. Available at this site: ftp://ftp.cs.berkeley.edu/ucb/sprite/papers/state.ps. *A rarely referenced discussion of distributed state; a broader perspective on the problems and challenges.*

[P+94] "NFS Version 3: Design and Implementation" by Brian Pawlowski, Chet Juszczak, Peter Staubach, Carl Smith, Diane Lebel, Dave Hitz. USENIX Summer 1994, pages 137-152. *The small modifications that underlie NFS version 3.*

[P+00] "The NFS version 4 protocol" by Brian Pawlowski, David Noveck, David Robinson, Robert Thurlow. 2nd International System Administration and Networking Conference (SANE 2000). *Undoubtedly the most literary paper on NFS ever written.*

[RO91] "The Design and Implementation of the Log-structured File System" by Mendel Rosenblum, John Ousterhout. Symposium on Operating Systems Principles (SOSP), 1991. *LFS again. No, you can never get enough LFS.*

[S86] "The Sun Network File System: Design, Implementation and Experience" by Russel Sandberg. USENIX Summer 1986. *The original NFS paper; though a bit of a challenging read, it is worthwhile to see the source of these wonderful ideas.*

[Sun89] "NFS: Network File System Protocol Specification" by Sun Microsystems, Inc. Request for Comments: 1094, March 1989. Available: http://www.ietf.org/rfc/rfc1094.txt. *The dreaded specification; read it if you must, i.e., you are getting paid to read it. Hopefully, paid a lot. Cash money!*

[V72] "La Begueule" by Francois-Marie Arouet a.k.a. Voltaire. Published in 1772. *Voltaire said a number of clever things, this being but one example. For example, Voltaire also said "If you have two religions in your land, the two will cut each other's throats; but if you have thirty religions, they will dwell in peace." What do you say to that, Democrats and Republicans?*

作业（测量）

在这个作业中，你将使用真实的跟踪记录（trace）做一点 NFS 跟踪分析。这些跟踪记录来自 Ellard 和 Seltzer 的工作 [ES03]。开始之前，务必阅读相关的 README，并（照例）从 OSTEP 作业页面下载相应的 tar 包。做这项分析时，awk 编程语言 [AKW88] 可能是个好玩的工具。

问题

1. 跟踪分析的第一个问题：利用第一列中的时间戳，确定这些跟踪记录是在哪个时间段采集的。这个时间段有多长？是哪一天/哪一周/哪一月/哪一年？（这和文件名给出的提示一致吗？）提示：用 `head -1` 和 `tail -1` 这两个工具取出文件的第一行和最后一行，然后计算。
2. 现在来统计一下操作的数量。跟踪记录中每种类型的操作各出现了多少次？按频率排序；哪种操作最频繁？NFS 名不虚传吗？
3. 现在更详细地看一些特定的操作。例如，GETATTR 请求会返回很多关于文件的信息，包括请求是为哪个用户 ID 执行的、文件大小等等。统计跟踪记录中被访问文件的大小分布；平均文件大小是多少？另外，跟踪记录中有多少个不同的用户访问了文件？是少数几个用户占了大部分流量，还是比较分散？GETATTR 的回复中还有哪些有趣的信息？
4. 你也可以查看针对某个文件的请求，确定文件是如何被访问的。例如，某个文件是被顺序读写的，还是被随机读写的？查看 READ 和 WRITE 请求/回复的细节来算出答案。
5. 流量来自许多机器，发往一台服务器（在这份跟踪记录中）。计算一个流量矩阵，给出跟踪记录中有多少个不同的客户端，以及发往每个客户端的请求/回复各有多少。是少数几台机器占了主导，还是比较均衡？
6. 利用时间信息和每个请求/回复的唯一 ID，你应该能算出某个请求的延迟。计算所有请求/回复对的延迟，并画出它们的分布。平均值是多少？最大值？最小值？
7. 有时请求会被重试，因为请求或它的回复可能丢失或被丢弃。你能在跟踪样本中找到这种重试的证据吗？
8. 通过更多的分析，你还可以回答很多其他问题。你认为哪些问题很重要？告诉我们，也许我们会把它们加到这里！

 © 2008–25, ARPACI-DUSSEAU